

ReClaw — Revive Your Old Smartphone with AI

PRISM Hackathon 2026 | Theme 2: Daily Utility (Smartphones)

PicoClaw (NDK) Qwen3-0.6B Multi-Runtime QAT Master-Slave

Problem Statement

Theme: Daily Utility (Smartphones)

- 1. **Intelligent Customizer** — Context-aware phone profiles (wallpaper, ringtone, volume, DND) based on location, time, and routine.
- 2. **Daily Commute Planner** — Calendar → real-time transit → departure alerts via Telegram/WhatsApp 30 min before.

5.3 billion old smartphones sit unused worldwide. ReClaw gives them a new brain.

Architecture

Master-Slave Model

Old phone = Slave (passive agent) — runs autonomously, senses context, executes skills.
Primary phone = Master — sends commands, receives alerts, overrides profiles via Telegram/WhatsApp.
No special app needed.

Component	Technology	Purpose
Agent Runtime	PicoClaw (NDK-compiled)	Native ARM binary, no Go/Termux dep
Language Model	Qwen3-0.6B (QAT)	On-device reasoning & NLU
Inference	Adaptive multi-runtime	Best backend per device
Fine-Tuning	PyTorch + TorchAO QAT	Framework-agnostic QAT
Advanced Mgmt	proot Linux distro	Full pkg mgmt without root

Adaptive Runtime Selection

No single-runtime lock-in. PicoClaw auto-selects based on device:

Runtime	Format	Best For	Min RAM
ExecuTorch	.pte	Snapdragon 6xx+	2GB
llama.cpp	.gguf	Broad compatibility	1.5GB
PicoLM (fork)	Custom	Ultra-constrained (<1GB)	512MB
LiteRT	.tflite	GPU delegate devices	1.5GB
MNN	.mnn	MediaTek/Kirin SoCs	1GB

QAT Fine-Tuning (Framework-Agnostic)

No vendor lock-in. Pure **PyTorch** + **TorchAO** QAT:

1. **Data Curation**: 1,700+ examples (6 categories, 30% Hinglish)
2. **QAT**: INT8 dynamic activation + INT4 weight, full fine-tuning, Colab T4
3. **Multi-Export**: .pte + .gguf + .tflite/.mnn from same checkpoint
4. **Deploy**: PicoClaw NDK binary auto-detects device → loads best runtime

Data Curation (1,700+ examples)

Category	Count	Example
Profile Switching	~400	"Switch to office mode" → [CALL device.volume.set(0)]...
Commute Queries	~300	"When should I leave?" → [CALL maps.directions...]
Device Commands	~350	"Set volume to 50" → [CALL device.volume.set(50)]
Context Sensing	~250	[SYSTEM] 9AM Monday Office → auto Office profile
Notification Mgmt	~200	"Summarize my messages" → digest
Master-Slave	~200	"Push sleep to old phone" → remote profile push

Pipeline: Manual seeds → Template expansion → LLM augmentation → Human validation

Key Features

Intelligent Customizer — Auto-profiles (Home/Office/Transit/Sleep) based on time + GPS. Personalized wallpaper, ringtone, volume packs.

Daily Commute Planner — Calendar → Maps API → departure alert 30 min before via Telegram.

Notification Hub — SLM summarizes messages, post-DND digest. Priority filtering.

Master-Slave — Old phone = passive agent. Primary phone controls via Telegram. Remote status, profile push, config updates.

User Stories

Story 1: Morning Commute — 7AM check calendar → 10AM meeting → 75min commute → Telegram alert at 8:15AM → auto Transit profile

Story 2: Master-Slave Push — "Push sleep mode to old phone" → remote activation → confirmed via Telegram

Story 3: Auto-Silent + Digest — GPS campus → DND → class ends → "8 msgs, 2 urgent from Mom"

Timeline

Phase	Dates	Deliverables
Phase 1	Apr 24	Proposal, architecture
Week 1	Apr 25–May 1	Data curation, QAT pipeline, NDK compile, proot setup
Week 2	May 2–8	SLM training, multi-export, skills, Telegram, master-slave, demo
Week 3	May 9–15	Polish, per-runtime benchmarks, documentation
Week 4	May 16–19	3 scenario videos, final report

Dependencies

Component	Purpose
PyTorch + TorchAO	QAT fine-tuning
ExecuTorch / llama.cpp / PicoLM / LiteRT / MNN	Multi-runtime inference
PicoClaw (NDK)	Agent runtime
Android NDK r26+	Native ARM compilation
proot-distro	Rootless Linux on Android

AI Usage Report

Category	Details
AI Model	Qwen3-0.6B (QAT fine-tuned)
Fine-Tuning	PyTorch + TorchAO QAT (INT8-INT4)
Runtimes	ExecuTorch / llama.cpp / PicoLM / LiteRT / MNN
Formats	.pte, .gguf, .tflite, .mnn
AI for Artifacts	Antigravity (docs), Colab (training)